

Arbre Rouge-Noir

Propriétés et implémentations

Antonin Ruan

29 mai 2025

1 Introduction

Les arbres rouge-noir (ou également arbre bicolore) sont des arbres binaires de recherche équilibrés, utilisé pour du stockage et de l'accès rapide a des informations triées. Par rapport à des arbres binaires de recherche standard, il possède une propriété de plus, leur couleur, qui permet de garantir un certain équilibrage de l'arbre. Ils étaient notamment au coeur du CFS, l'ordonnanceur du noyau Linux de 2007 à 2023. On s'intéresse alors à leurs propriétés et on propose une implémentation partielle en C. Celle-ci étant très largement inspiré de l'article Wikipédia *Red-black Tree* [1]

2 Propriétés

Les arbres bicolores étant construits à partir des arbres binaires de recherche, ils en respectent les propriétés, et leur structure est similaire.

Dans un arbre de recherche, chaque nœud est constitué de 3 éléments : 2 nœuds fils potentiellement nuls que l'on nomme `left` et `right` et une valeur numérique `key` permettant d'ordonner l'ensemble des nœuds. L'ordre est assuré de la manière suivante, pour n'importe quel nœud `A` donné, toutes les valeurs présentes dans le sous arbre `A.left` sont strictement inférieures à `A.key` et symétriquement toutes les valeurs présentes dans le sous arbre `A.right` sont supérieures ou égales à `A.key`.

Dans le cas des arbres bicolores, on ajoute un nouvel élément à chaque nœud : sa couleur et les propriétés suivantes doivent être respectés par l'arbre :

- (i) Chaque est soit rouge, soit noir
- (ii) Les nœuds vides (les feuilles) sont considérés noir
- (iii) Un nœud rouge ne peut pas avoir de fils rouge
- (iv) Pour tous les nœud, les chemins à chacune de ses feuilles passent par un même nombre de nœud noir.

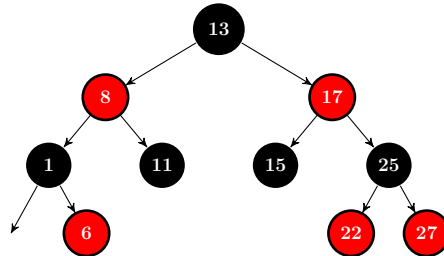


FIGURE 1 – Exemple d'arbre bicolore valide

3 Implémentation

On pourra s'aider tout au long du document, en plus des visualisations présentes, du site de l'Université de San Francisco[2], pour observer plus en détails les opérations sur les arbres.

Les opérations de recherche et de parcours ne modifiant pas l'arbre, ne seront pas détaillées ici, seule les opérations d'insertion et de suppression peut mener à des violations des contraintes 2.

3.1 Notations

Pour l'ensemble l'implémentation, on s'appuiera sur les structures de données suivantes :

```
typedef enum {RED, BLACK} color;
typedef enum {LEFT, RIGHT} dir;

typedef struct node node;
struct node {
    node *parent;
    union { // We can either use ->left or ->child[0]
        struct {
            node *left;
            node *right;
        }
        node *child[2];
    };
    color color;
    int key;
};

typedef struct {
    node *root;
    node *left;
} rb_tree;

#define DIR(n) (n == n->parent->left ? LEFT : RIGHT)
```

3.1.1 N-Hauteur

On appelle la N-hauteur (ou hauteur noire) d'un arbre, le nombre de nœuds noirs présent sur le chemin entre la racine de l'arbre et une de ses feuilles (constant d'après (*iv*)). On la notera h_N ou h_{noir} . De manière plus précise on notera $h_N(N)$ la N-hauteur du sous arbre dont la racine est N.

3.1.2 Rotations

On définit également 2 opérations sur nos arbres, `rot_left` et `rot_right`. Le détail de leur implémentation est en annexe 4.1. Leur effet est décrit dans les figures 2 et 3.

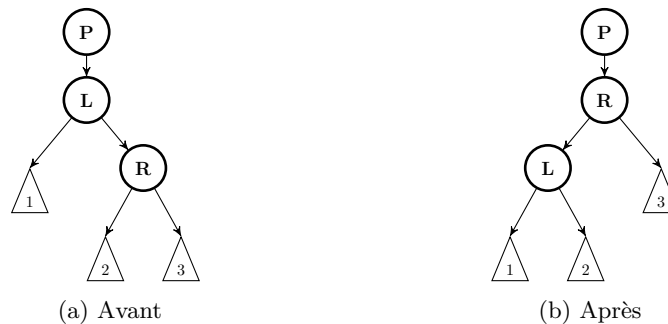


FIGURE 2 – Rot Left(L)

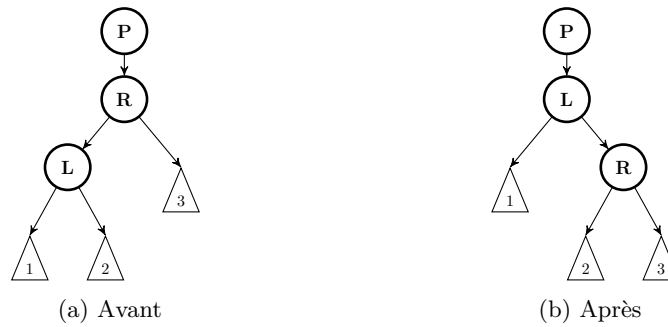


FIGURE 3 – Rot Right(R)

3.1.3 Direction et nœuds voisins

De manière générale, lorsqu'on parlera d'un nœud N , on parlera de sa direction dir défini comme valant **LEFT** (respectivement **RIGHT**) si N est le fils gauche (resp. droit) de son parent. Cette notion n'ayant évidemment pas de sens dans le cas où N est la racine de l'arbre.

Les opérations à effectuer seront généralement dépendantes de ces autres nœud :

- son parent, noté P
- son grand-parent, noté G
- son frère, noté S
- son oncle, noté U
- son cousin proche, noté C , le fils de S ayant la même direction que N
- son cousin éloigné, noté D , le fils de S , ayant la direction opposée à N

Il est à noter que l'ensemble de ces nœuds peuvent être nul selon la configuration de l'arbre. Dans la suite on considère un nœud nul comme noir.

On représente en figure 4, N et ses voisins en supposant sa direction est **LEFT**.

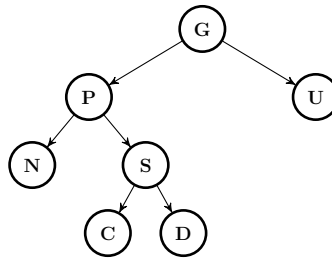


FIGURE 4 – N et ses voisins

3.1.4 Notes sur les illustrations

Les diagrammes illustrant les différents cas d'insertion et de suppression sont tous composés d'au plus 4 cas

- Entrée : l'état de l'arbre avant le traitement du cas
- Rotation : si nécessaire, l'effet de la rotation du cas sur l'arbre
- Couleur : si nécessaire, le changement de couleur effectué
- Notation : si nécessaire, l'arbre après les changements de notation

3.2 Insertion

L'insertion d'une nouvelle valeur v dans l'arbre commence tout d'abord par trouver la feuille qui permet d'insérer v dans l'arbre tout en respectant ces contraintes d'arbre de recherche. Ce qui correspond en fait à faire une recherche de v . Une fois cette feuille trouvée, on la remplace par un nœud

$N = \text{node}\{\text{.key} = v, \text{.color}=\text{RED}\}$. Le nœud est temporairement colorié en rouge de manière et ne pas violer la contrainte (*iv*). On va donc remonter dans l'arbre à partir de N tout en corrigeant les potentielles violations de propriétés. On a donc une boucle qui les invariants suivants :

- N est le nœud courant, le nœud inséré pour la première itération
- N est rouge au début de l'itération
- (*iii*) est satisfait pour toutes les paires enfants parents avec comme seule exception possible la paire (N, P)
- Le reste des propriétés sont satisfaites

A chaque itération, on s'appuie sur la table 1 qui donne une disjonction des cas possibles à traiter et la manière de les résoudre.

Avant				Cas	Après				Next
P	G	U	x		P	G	U	x	
●				Cas 1					✓
●	●	●		Cas 2					
⁻¹				Cas 3					✓
●	⁻¹			Cas 4	●				✓
●	●	●	ⁱ²	Cas 5	●	●	●	^{e2}	Cas 6
●	●	●	^{e2}	Cas 6	●	●	●	x	✓

TABLE 1 – Diagramme d'insertion

¹ - Indique que le nœud est nul

² i et e indique que N est configuration intérieur ou extérieur, voir annexe 4.2

Une fonction permettant l'équilibrage de ces différents cas une fois la recherche et l'insertion effectué est proposé à l'annexe 4.3

3.2.1 Cas 1

P est déjà noir la contrainte (*iii*) est donc respecté, notre arbre est valide. On s'arrête

3.2.2 Cas 2

P et U sont tous deux rouges, G est donc nécessairement noir car (*iii*) pour la paire (U, G) . Sans violer la contrainte (*iv*) on peut alors colorer P et U en noir et G en rouge. En notant désormais G , comme N le nœud d'intérêt l'invariant de boucle est respecté et on continue d'itérer.

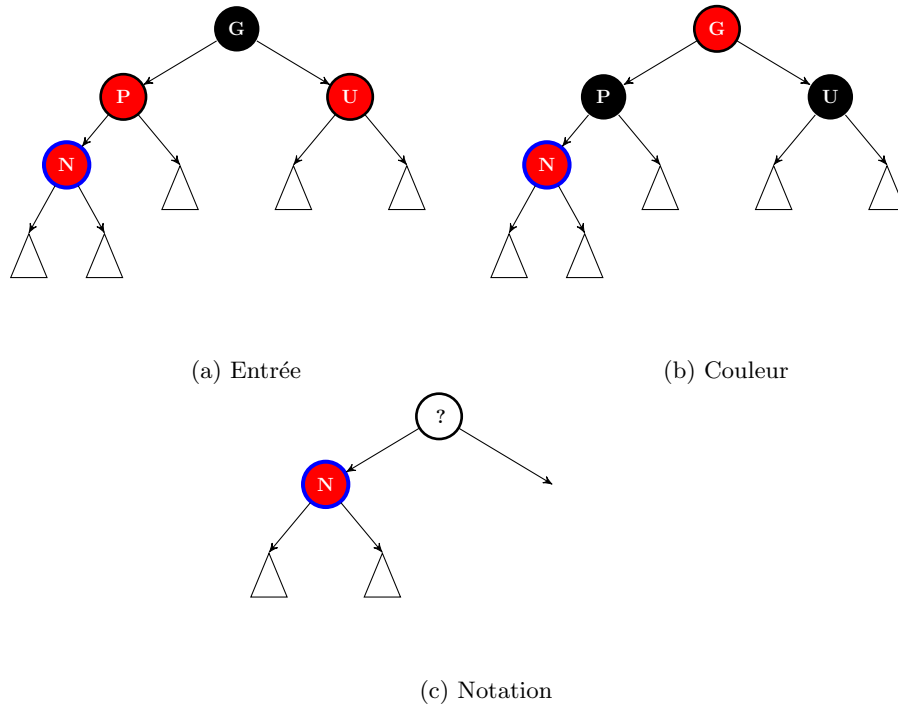


FIGURE 5 – Cas 2 d'insertion

3.2.3 Cas 3

N est la racine de l'arbre, rien à faire

3.2.4 Cas 4

P est rouge et est la racine de l'arbre, on peut simplement changer la couleur de P à noir sans violer aucune contrainte.

3.2.5 Cas 5

P est rouge mais U est noir, le but est de se ramener dans une situation similaire au Cas 2. On veut alors se ramener au Cas 2 par une opération de rotation tout en gardant N comme fils de P. Ceci n'est possible que si N est en configuration extérieur, ce qui constitue le Cas 6. Pour s'y ramener on effectue une première rotation entre N et P, ce qui échange le rôle de P et N. Comme ils sont tout deux de même couleur, on préserve (iv) et après le changement de notation adapté on se retrouve bien dans le Cas 6

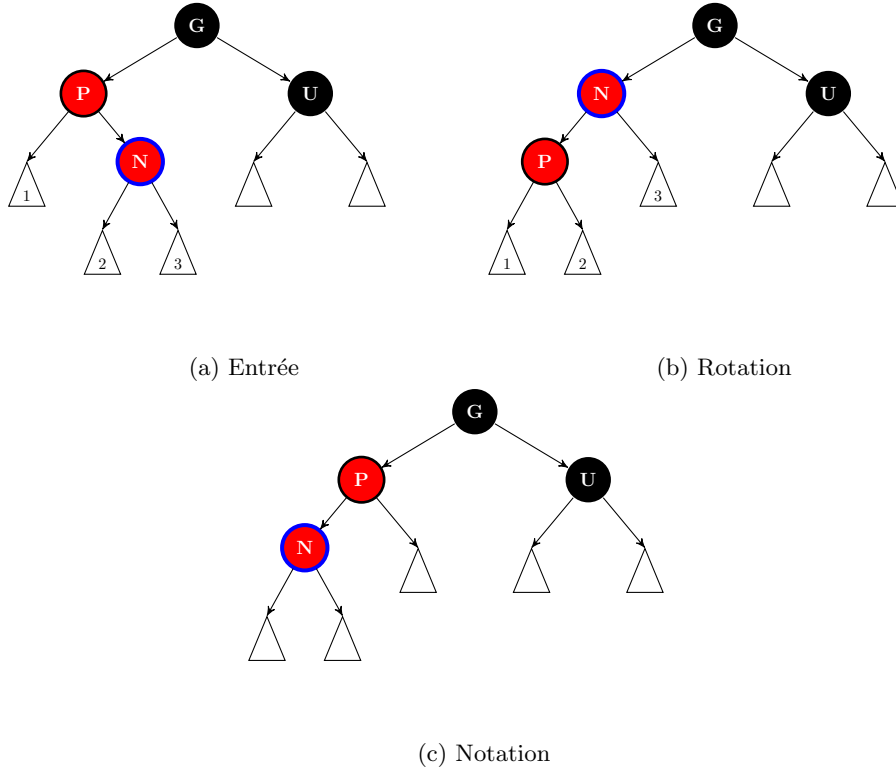


FIGURE 6 – Cas d'insertion 5

3.2.6 Cas 6

On peut désormais effectuer une rotation de P et G. On peut alors colorier P en noir et G en rouge. Ce qui permet de résoudre la violation à (iii). Et quand l'ensemble des chemins par G quand il était noir passe désormais par P, (iv) reste satisfait. L'arbre est désormais valide on s'arrête.

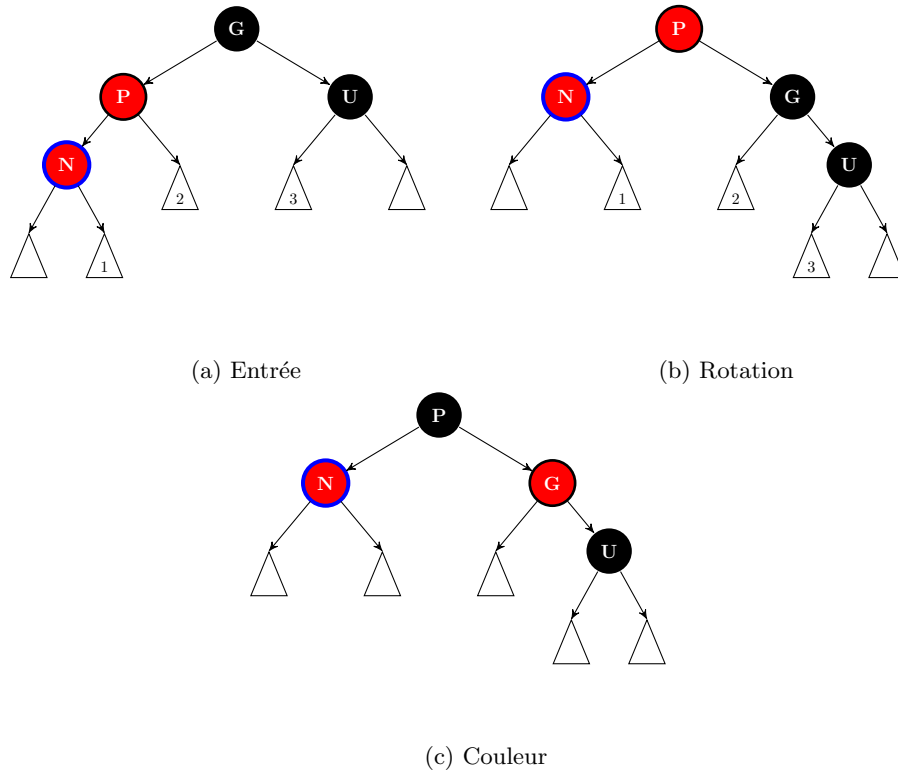


FIGURE 7 – Cas d'insertion 6

3.3 Suppression

La suppression commence l'insertion : on cherche déjà le nœud contenant la valeur à supprimer. Une fois qu'on l'a trouvé on distingue quelque chose à traiter avant de s'attaquer au gros du problème.

3.3.1 Cas simples

- Si N a 2 enfants (non nul), on échange la valeur de N avec celle du nœud N' le plus à droite du sous arbre $N \rightarrow \text{left}$. Sa valeur étant la plus à droite du sous arbre $N \rightarrow \text{left}$, elle permet bien de préserver les propriétés d'arbre de recherche. On supprime alors le nœud N' , qui étant le plus à droite d'un sous arbre a au maximum 1 enfant non nul. On continue donc sur les cas suivants
- Si N a un unique enfant, la propriété (iv) assure que cet enfant est de couleur rouge. On peut alors échanger les 2 valeurs et supprimer l'enfant, ce qui conclut la suppression
- Si N n'a pas d'enfant et que c'est la racine de l'arbre, on remplace la racine par une valeur nul, l'arbre devient vide.
- Si N n'a pas d'enfant et est rouge on peut le supprimer sans violer de contrainte.
- Si N n'a pas d'enfant et est noir, sa suppression va violer la contrainte (iv) et créer un déséquilibre, on traite de ces cas la partie suivante.

3.3.2 Suppression d'une feuille noire

De la même manière que pour l'insertion on va remonter l'arbre pour corriger le déséquilibre créé par la suppression. Ce qui mène à une boucle avec les invariants suivants :

- N est noir.
- Le nombre de nœuds noirs sur les chemins passant par N est diminué de 1 par rapport à avant la suppression. Il est inchangé sur tous les autres. On a donc une violation de (iv) s'il existe de tel autre chemin.
- Les autres propriétés sont satisfaites

On se réfère alors à la table 2 pour une disjonction des cas possibles

Une fonction pour la gestion des cas présents dans le tableau est proposée dans l'annexe 4.4

Avant				Cas	Après				Next
P	S	C	D		P	S	C	D	
-				Cas 1					✓
●	●	●	●	Cas 2	?				?
●	●	●	●	Cas 3	●	●	?	●	?
●	●	●	●	Cas 4	●	●	●	●	✓
●	●	●	●	Cas 5	●	●		●	Cas 6
●	●		●	Cas 6	●	●		●	✓

TABLE 2 – Diagramme de suppression

3.3.3 Cas 1

N est la racine, tous les chemins passent par N l'arbre est valide, on s'arrête

3.3.4 Cas 2

P, S et les fils de S sont tous noirs, en coloriant S en rouge, on décremente de 1 $h_N(S)$ et alors $h_N(N) = h_N(S)$. En notant alors P N, on vérifie de nouveau les invariants de la boucle, on continue d'itérer.

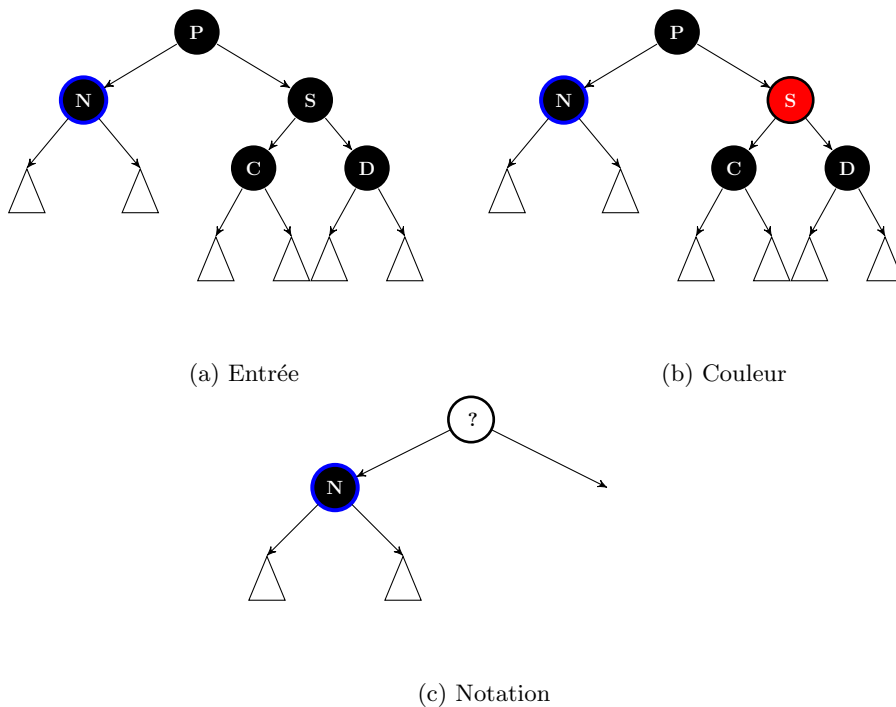


FIGURE 8 – Cas de suppression 2

3.3.5 Cas 3

S est rouge, donc nécessairement P, C et D sont noirs, une rotation entre S et P transforme S en le grandparent de N. On échange les couleurs de S et P de manière à conserver l'invariant. On doit toujours incrémenter $h_N(N)$ pour équilibrer l'arbre mais maintenant après les changements de notation adapté, P est rouge. On se retrouve dans l'un des cas suivants.

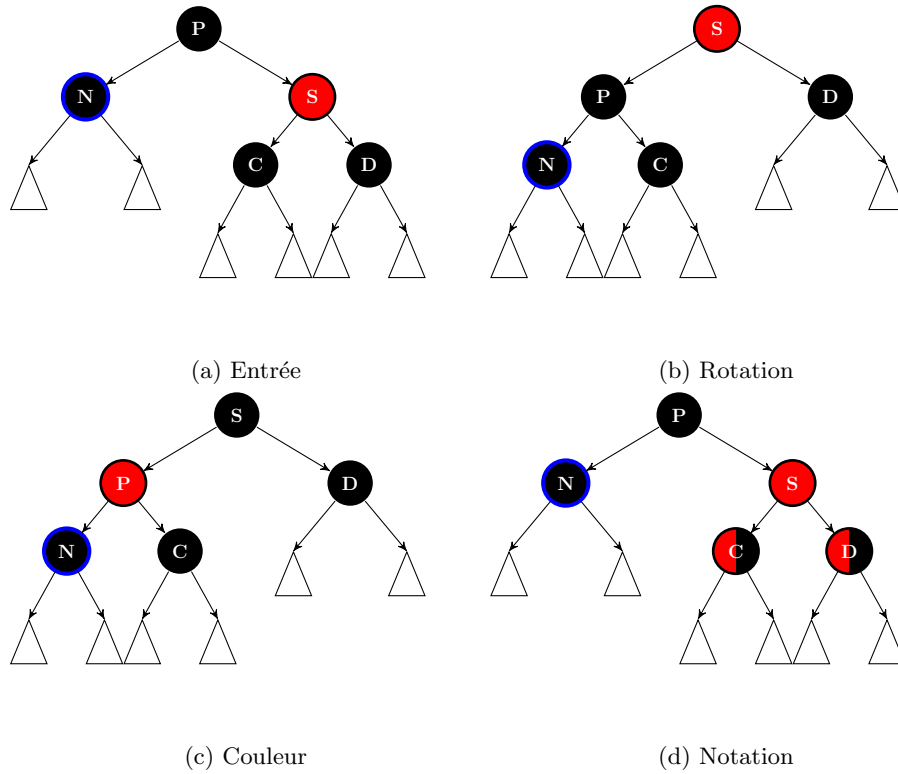


FIGURE 9 – Cas de suppression 3

3.3.6 Cas 4

S et ses fils sont noirs, mais P est rouge. On peut échanger les couleurs de S et P ce qui permet d'avoir $h_N(N) = h_N(S)$. Et donc un arbre valide ce qui permet de s'arrêter.

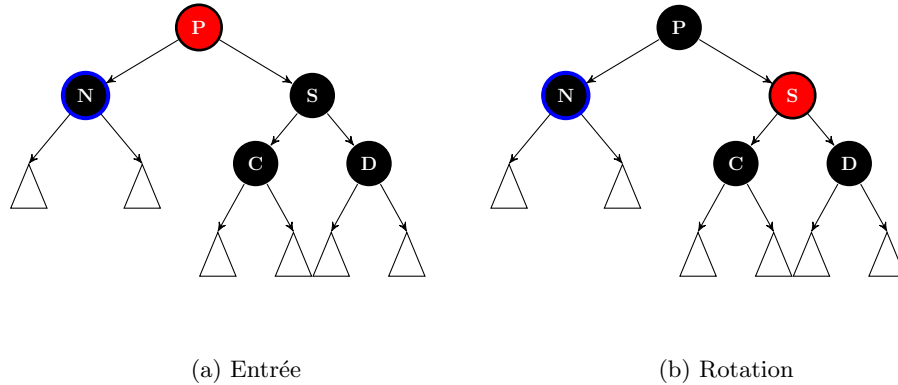


FIGURE 10 – Cas de suppression 4

3.3.7 Cas 5

S et D sont noirs, C rouge. Une rotation entre S et C transforme C en le nouveau frère de N. On échange les couleurs de S et C pour le nombre de nœuds noir dans tous les chemins. On se retrouve alors dans la situation du Cas 6

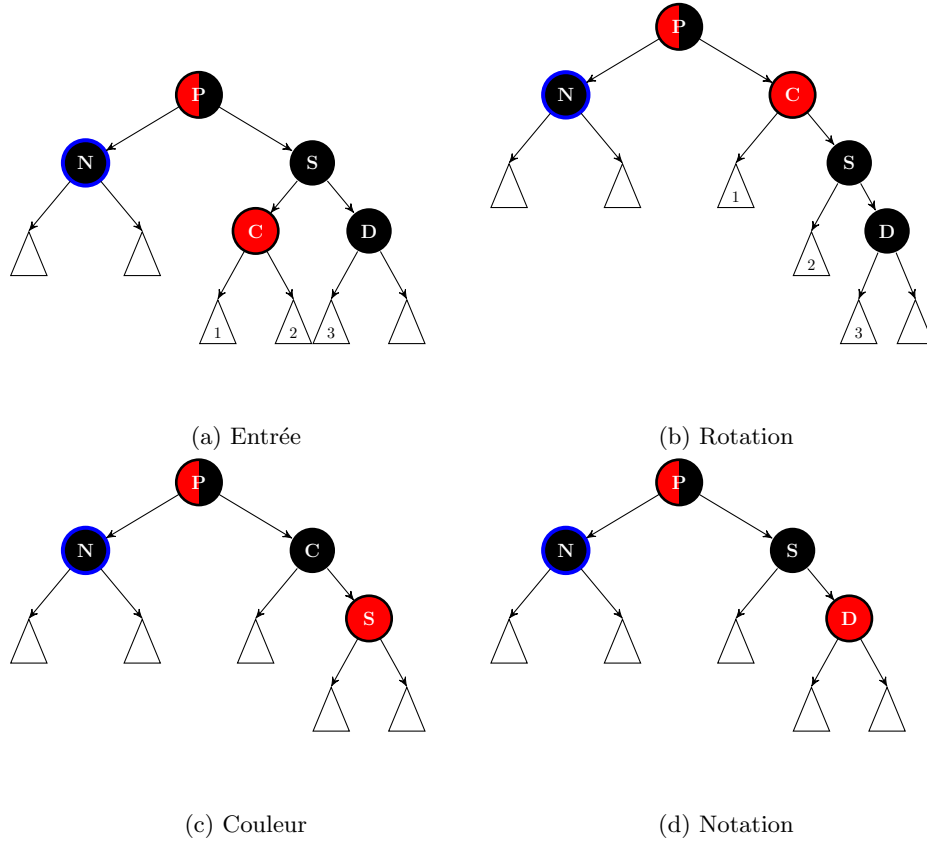
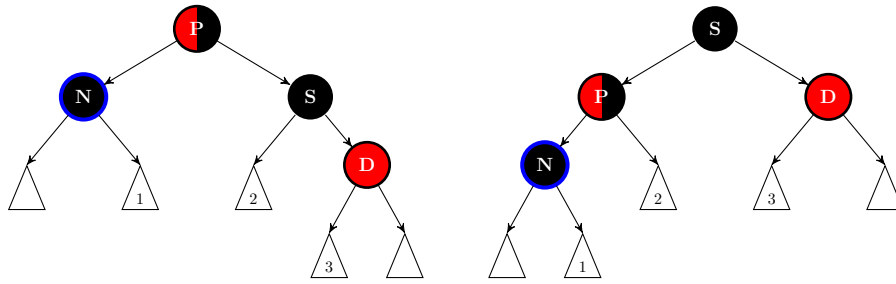


FIGURE 11 – Cas de suppression 5

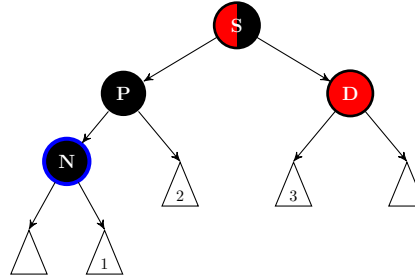
3.3.8 Cas 6

S est noir et D est rouge. Après une rotation entre S et P, S est devenu le parent de P et D. On inverse les couleurs de P et S et on colorie D en noir. Le sous arbre conserve sa couleur ce qui assure que (iii) est préservée. Les chemins passant par D et 3 ont toujours le même nombre de nœuds noirs. Et ceux passant par N ont incrémenté de 1 (soit P est devenu noir, soit P était déjà noir et S a été ajouté comme grandparent noir). Notre arbre est donc valide et on peut s'arrêter.



(a) Entrée

(b) Rotation



(c) Couleur

FIGURE 12 – Cas de suppression 6

Références

- [1] “Red-black tree.” https://en.wikipedia.org/wiki/Red-black_tree#, 2025.
- [2] “Red/black tree visualization.” <https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>.

4 Annexes

4.1 Implémentation de rot_left et rot_right

```
void rotate(rb_tree *t, node *n, dir dir) {
    node *p = n->parent;
    node *root = n->child[1 - dir];
    node *child = root->child[dir];

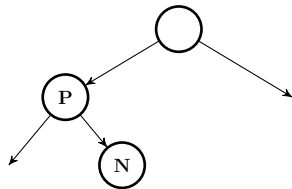
    n->child[1 - dir] = child;
    if (child) child->parent = n;

    root->parent = p;
    if (p)
        p->child[DIR(n)] = root;
    else
        t->root = root;

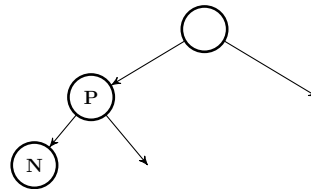
    n->parent = root;
    root->child[dir] = n;
}
```

4.2 Nœud intérieur et extérieur

On appelle **i** la configuration où **N** et **P** ont une direction différentes et **e** celle où ils possèdent la même direction.



(a) Configuration **i**



(b) Configuration **e**

FIGURE 13 – Exemple de configuration

4.3 Balance insertion

```
void balance_tree_insert(rb_tree *t, node *n, node *parent) {
    // n is the root of tree
    if (!parent) {
        n->color = BLACK;
        return;
    }

    do {
        // Cas 1
        if (parent->color == BLACK) return;

        node *gp = parent->parent;

        // Cas 3
        if (!gp) {
            parent->color = BLACK;
            return;
        }

        dir dir = DIR(parent);
        node *uncle = gp->child[1 - dir];
        if (!uncle || uncle->color == BLACK) {
            if (n == parent->child[1 - dir]) {
                // Cas 5
                rotate(t, parent, dir);
            }
        }
    } while (1);
}
```

```

        n = parent;
        parent = gp->child[dir];
    }

    // Cas 6
    rotate(t, gp, 1 - dir);
    gp->color = RED;
    parent->color = BLACK;
    return;
}

// Cas 2
parent->color = BLACK;
uncle->color = BLACK;
gp->color = RED;
n = gp;

} while ((parent = n->parent));

return;
}

```

4.4 Balance suppression

```

void balance_tree_remove(rb_tree *t, node *n) {
    node *p;
    node *s;
    node *c;
    node *d;

    do {
        dir = DIR(n);
        p = n->parent;
        s = p->child[1 - dir];
        c = s->child[dir];
        d = s->child[1 - dir];

        // Cas 3
        if (s->color == RED) {
            rotate(t, p, dir);
            p->color = RED;
            s->color = BLACK;
            s = c;
            c = s->child[dir];
            d = s->child[1 - dir];
        }

        if (!d || d->color == BLACK) {
            if (!c || c->color == BLACK) {
                s->color = RED;
                if (p->color == BLACK) {
                    // Cas 2
                    continue;
                } else {
                    // Cas 4
                    p->color = BLACK;
                    return;
                }
            } else {
                // Cas 5
                rotate(t, s, 1 - dir);
                c->color = BLACK;
                s->color = RED;
                d = s;
                s = c;
            }
        }

        // Case 6
        rotate(t, p, dir);
        s->color = p->color;
    }
}

```

```
        p->color = BLACK;
        d->color = BLACK;
        return;
    } while ((n = n->parent) && n->parent);

    // Cas 1
    t->root = n;
    return;
}
```